Track 1: Smart Document Generator

Objective

Build a reusable system that **automates the end-to-end document creation workflow** by converting event/activity requirements into standardized, editable club documents using predefined templates. The system should minimize manual data entry, template editing, formatting, and document preparation across recurring club activities.

Mandatory Requirements

The solution must:

- Accept input through forms and/or natural language.
- Extract relevant information and identify/recommend the required document type.
- Support multiple document types and customizable templates.
- Generate documents in DOCX and PDF while preserving template structure and formatting.
- Maintain consistent club branding such as logos, colors, fonts, headers, and footers.
- Detect missing information and ask the user instead of fabricating official details.
- Allow users to review and edit generated documents.
- Keep information consistent throughout the document.
- Use an LLM/intelligent extraction mechanism as a fallback for natural-language or unstructured input.
- Allow the club to add or modify templates without major code changes.

Good to Have

- Automatic document-type detection
- Conditional/dynamic sections
- Batch generation of multiple documents
- Version history
- Approval/review workflow
- Reusable clauses and sections
- Updating existing documents when source information changes
- Template preview/editor
- Role-based access control
- Google Drive/Sheets integration
- Email distribution
- Digital signatures
- Usage analytics
- Multilingual support

Example

“We are conducting a two-day Web Development Workshop on 14–15 October in Seminar Hall for around 120 students.”

The system should extract the information, identify missing details, and generate relevant documents such as an event proposal and permission letter using the club's templates and branding.

What NOT to Do

- ❌ Build a simple ChatGPT/Gemini wrapper that only generates text.
- ❌ Hardcode a single template.
- ❌ Generate documents with inconsistent/random formatting.
- ❌ Fabricate missing official information.
- ❌ Make users manually copy AI output into Word/Docs.
- ❌ Build a generic chatbot where document generation is secondary.
- ❌ Hardcode every possible document/event combination.
- ❌ Focus on UI while keeping the actual workflow manual.

Evaluation Focus

Solutions will be evaluated on practical club utility, automation, document quality, accuracy, template flexibility, technical implementation, extensibility, usability, and reliability.

The final solution should be something the club can realistically adopt and continue using after the event.

Example docs:
📄 EXEC MEET_public.pdf
📄 Bill Certificate_public.pdf
📄 BIll_summary_public.pdf

Track 2: Conflict-Aware Events and Expenses Manager

Objective

Build a centralized management system for the ACM core that provides campus-wide event visibility to prevent scheduling clashes and maintains transparent fund tracking, acting as a digital institutional memory for current and future teams.

Mandatory Requirements

The solution must:

- Maintain a centralized campus event calendar where ACM executives and representatives from other clubs can input or submit event details (name, date, time, venue, type, expected audience, and organizer) for ACM core verification.
- Automatically check and flag potential scheduling overlaps or conflicts when a proposed ACM event clashes with other campus activities targeting the same audience.
- Provide an expense and fund management module to record financial transactions with details like amount, category, date, associated event, description, and uploaded receipts/bills.
- Automatically compute and display monthly or semester expenditures, remaining budgets, and category-wise spending breakdowns.
- Act as an institutional memory, storing historical data on past events, associated expenses, spending patterns, and scheduling conflicts for succeeding executive teams.
- Use the specified tech stack: Frontend (Next.js, Tailwind CSS, Shadcn UI, FullCalendar), Backend (Node.js, Express.js or Next.js API Routes), Database & ORM (PostgreSQL, Prisma), Storage (Cloudinary / AWS S3), and Authentication (NextAuth.js).

Good to Have

- Role-based access control to differentiate between regular members, club executives, and ACM core verifiers.
- Visual dashboard indicators showing total spent, remaining budget, and dynamic calendar conflict warnings.
- Automated notifications or early alerts for potential overlaps to empower the core team to make informed scheduling decisions.
- Secure digital storage for bills and receipts linked directly to specific events and expense categories.

Example

"We are planning a technical workshop from 5 to 7 PM, and another club already has a major technical event during the same period." The system should flag it as a potential conflict, display

the expenditure breakdown against a ₹50,000 semester budget, and provide the core team with clear visibility to make adjustment decisions.

What NOT to Do

- ❌ Rely on distributed spreadsheets, separate bills, messages, or individual records for expense tracking.
- ❌ Depend entirely on informal WhatsApp or Instagram channels for campus event visibility.
- ❌ Force future executive teams to start from scratch without access to historical event and financial data.
- ❌ Let the system make automated cancellation decisions instead of leaving choices to the core team.

Evaluation Focus

Solutions will be evaluated on practical club utility, conflict-detection accuracy, transparency of fund management, institutional memory retention, technical implementation, and usability. The final solution should be something the club can realistically adopt and continue using long-term.

Track 3: Team Recruitment Automation Tool

Objective

Build a reusable, tunable tool that helps hackathon organizers shortlist the top X teams from a large pool of registrations. Each team's members submit whichever profile links and documents they have (resume, GitHub, LinkedIn, portfolio). The tool should turn this data into a fair, explainable shortlist while staying flexible enough to fit different hackathons, team sizes, and selection philosophies.

Mandatory Requirements

The solution must:

- Provide a frontend GUI for organizers to load data, configure selection criteria, run the shortlisting, and review results.
- Accept team data in tabular (CSV, e.g. Google Forms exports) and/or JSON format. Building data-collection forms is not required.
- Handle teams of any size, from 1 member to many, and document how individual signals are combined at team level.
- Handle partial data gracefully. Teams may provide only some of the resume, GitHub, LinkedIn, and portfolio links, and missing data must not crash the tool or be silently scored as zero without the organizer's knowledge.
- Select the top X teams out of N, where X is configurable.
- Support at least one selection strategy (weighted scoring) plus configurable hard filters or eligibility rules (e.g. minimum GitHub contributions, required achievements, mandatory profile fields).
- Be tunable through configuration (UI controls and/or config files) rather than code changes: weights, thresholds, rules, and X should all be adjustable.
- Remain unopinionated. No fixed criteria should be hardcoded as "the right way" to judge teams.
- Explain results, showing why a team was shortlisted or rejected, with a score breakdown or rule-level reasoning.
- Export the final shortlist (e.g. CSV/JSON) along with the configuration used.
- Use a modular, maintainable codebase so that new criteria, data sources, or scoring methods can be added easily.
- Be reliable and reproducible: the same data and configuration should give the same output. If AI or LLMs are used, their outputs must be validated, and uncertain or low-confidence results must be flagged, not presented as fact.

Good to Have

- Pairwise/head-to-head comparison mode alongside scoring
- Live re-ranking as weights or rules change
- Saved configuration presets and a comparison between runs

- GitHub API integration (contributions, repo quality, languages, activity)
- Resume parsing (rule-based and/or LLM-assisted)
- Portfolio and LinkedIn analysis (via user-provided exports or text, within legal limits)
- Detection of suspicious or padded profiles (empty or forked repos, duplicate submissions, copied resumes)
- Manual overrides such as pin, exclude, or waitlist
- Tie-breaking rules
- Anonymized or blind review mode and bias-reduction options
- Sensitivity analysis, showing how stable the shortlist is under small changes in the weights
- Diversity and skill-coverage balancing
- Audit log of decisions and configuration changes
- Efficient performance on large datasets (thousands of teams)
- Team-size normalization to keep solo participants and large teams on a fair footing

Example

An organizer uploads a Google Forms CSV with 300 teams (sizes 1 to 5), each with some subset of resume, GitHub, LinkedIn, and portfolio links. They set X = 50, require at least one member with 50+ GitHub contributions in the past year, and give higher weight to projects and achievements than to education. The tool produces a ranked shortlist of 50 teams with a per-team score breakdown, flags teams with incomplete data, and lets the organizer change the weights and see the shortlist update.

What NOT to Do

- ❌ Build a simple ChatGPT/Gemini wrapper that just "ranks" teams with no transparency.
- ❌ Hardcode a single scoring formula or fixed criteria with no way to tune it.
- ❌ Assume every team has every profile link or the same team size.
- ❌ Give unexplained scores or a black-box shortlist.
- ❌ Silently discard or penalize teams with missing data without telling the organizer.
- ❌ Fabricate or infer profile information that is not in the data or the fetched sources.
- ❌ Require code changes to adjust weights, thresholds, or rules.
- ❌ Focus on UI polish while the actual selection logic is shallow.

Evaluation Focus

Solutions will be evaluated on flexibility and configurability, quality and clarity of explanations, practical utility and features, reliability (including any AI components), handling of edge cases (missing data, varied team sizes), code modularity and maintainability, and usability. The final solution should be something the club can realistically adopt for future hackathons.

Track 4: Content and Marketing Studio

Objective

Build an intelligent design tool that helps club designers and organizers produce print-ready publicity and identity material quickly. The tool takes a natural-language description of what is needed, gathers context and relevant artifacts (logos, event details, participant data), asks the right follow-up questions, and generates 2-3 design drafts to choose from.

The **benchmark use case is ID cards** for hackathon participants, organizers, mentors and volunteers, or for workshop attendees. Teams must master this niche first. Only after the mandatory requirements are met may they extend the tool to other design types such as certificates, posters, and banners.

Mandatory Requirements

The solution must:

- Provide a frontend GUI for the full workflow: describing the requirement, answering questions, reviewing drafts, and exporting.
- Accept a natural-language request (e.g. "ID cards for our 24-hour hackathon, with separate looks for participants, organizers and mentors") and use it, along with any provided context and artifacts, to drive the workflow.
- Ask relevant follow-up questions for missing information (event name, dates, venue, roles, fields to show, etc.) instead of fabricating details. The questioning approach (fixed, adaptive, conversational, form-based, or any other UX) is up to the team.
- Generate **2-3 distinct design drafts** for the user to choose from.
- Support multiple ID card roles or categories (participant, organizer, mentor, volunteer, etc.) with clear visual differentiation between them.
- Produce output that is **high resolution and suitable for printing**. Vector or high-DPI export is expected, and the design must remain sharp at physical card sizes.
- Support more than one export format (e.g. PDF, PNG, SVG) and let the user switch between them or tune export settings such as size, DPI, and orientation.
- Include the **ACM NIT Surat and SVNIT logos** in the default MVP output, with correct placement, proportions, and no distortion.
- Populate cards using real, provided data (names, roles, photos, IDs, etc.) and keep information accurate and consistent across the design.
- Use a modular, maintainable codebase so new design types, templates, and fields can be added without major rewrites.
- Be reliable: outputs must be consistent and usable without manual fixing. If AI is used, its outputs must be validated (e.g. no overflowing text, broken layouts, or invented details).

Good to Have

- Batch generation of cards from a CSV/JSON (e.g. a Google Forms export), including photos and QR codes/barcodes, with print sheets placing multiple cards per A4/A3 page
- Editable drafts through an in-browser editor (move, resize, edit text, swap colors)
- Editable, reusable templates, including adding new templates without code changes
- Tunable brand kit (logos, colors, fonts) applied automatically, and switchable per event
- Print-ready features such as bleed, crop marks, and safe margins
- QR code/barcode generation for check-in, attendance or verification. (link would be of the acm website attendance feature, but for demo/testing dummy endpoints may be used)
- Automatic layout adaptation for long names, missing photos, or missing fields
- Design variations across style, color, and layout with regenerate and refine options
- Refinement through natural language (e.g. "make it more minimal", "move the logo to the top")
- Extension to other design types: certificates, posters, banners, and social media creatives, using the same template and brand system
- Version history and saved projects
- Preview of the card in a realistic mockup (lanyard, badge holder)

Example

"We are hosting a 24-hour hackathon on 18 October at SVNIT with about 200 participants, 20 mentors and 15 organizers." The tool identifies the context, asks for missing details (event name, tagline, venue, fields to show, photo availability), and generates 3 ID card drafts with role-based color coding and both logos. The user picks one, tweaks it if needed, and exports print-ready files.

What NOT to Do

- ❌ Build a simple wrapper around an image-generation model that outputs unusable, non-editable, low-resolution pictures.
- ❌ Hardcode a single ID card design with no way to extend it.
- ❌ Generate cards with garbled text, distorted logos, or inconsistent layouts.
- ❌ Fabricate event details, names, or other official information.
- ❌ Export low-resolution images that cannot be printed.
- ❌ Make users manually assemble the final design in another tool.
- ❌ Build a generic chatbot where the actual design generation is secondary.
- ❌ Jump to posters and banners before the ID card workflow is solid.

Evaluation Focus

Solutions will be evaluated on the quality and print-readiness of output, accuracy and consistency of generated designs, effectiveness of the requirement-gathering flow, usability, reliability (including any AI components), modularity and extensibility, and design variety. Extension to other design types is treated as a bonus only after the ID card benchmark is fully met. The final solution should be something the club can realistically adopt for future events.

Track 5: Sponsy Bot - Sponsorship Outreach Automation

Objective

Build a human-in-the-loop sponsorship outreach system that helps the club find, prioritize, contact and follow up with potential sponsors. The system should research companies and decide what to ask each one for. It should also draft personalized emails and track every conversation and follow-up. An organizer approves each outgoing action with a single click. It replaces scattered spreadsheets, forgotten follow-ups and generic mass emails.

Mandatory Requirements

The solution must:
- Maintain a sponsor tracker that records each company, its category or tier, current status (not contacted, contacted, follow-up, interested, declined to be contacted by them, and the full email history.
- Research each company and use its profile and past sponsorship history to prioritize and categorize sponsors. Recommend what to ask each sponsor for (moructure/credits) based on the company's history, and show the reasoning.
Generate company-specific, personalized email sent to everyone.
- Keep a human in the loop: no email is sent without an organizer's approval, and the organizer can edit a draft before approving it.
- Send approved emails, track replies, and classify each as interested, no reply, or declined.
- Schedule follow-ups automatically for sponfixed limit (e.g. max 2 follow-ups, thenstop), so no sponsor is ever spammed.
- Move interested sponsors to the next step,ritual meeting.
- Prevent duplicate outreach, so two team members never contact the same company.
- Use only free APIs and free tiers, and mantas smartly.
- Handle an expected volume of 20–40 outgoing emails per day within those limits.
- Avoid fabricating company details, past option. Unverified information must be flagged, not presented as fact.

Suggested Tech Stack
- Workflow: LangGraph
- Verification / research: Tavily
- Contact search: Apollo / Hunter
- AI model: Gemini or Grok (free API)

Teams may swap in other free alternatives.

Good to Have
- Automatic contact discovery: finding the r at each company
- One-click automation, where a single approval triggers the whole next step, from sending follow-ups to confirming virtual meetings

- Dashboard showing pipeline stages, response rates, and money or benefits secured
- Meeting booking through a calendar link or
- Sponsorship brochure or deck attached, tailored to each tier
- Reusable email templates for first contact ls
- Google Sheets integration for the tracker
- Reply summaries and suggested responses
- Notifications when a sponsor replies
- Institutional memory: past sponsors, what , saved for future teams

Example

"We need sponsors for our 24-hour hackathon 1,50,000 plus swag, and we have a list of 60 companies."

The system researches each company and splits them into tiers (cash, goodies, infrastructure credits). It drafts personalized emails for the organizer to r emails a day. Companies that don't reply get up to two follow-ups, companies that decline are marked closed, and a call is booked with each interested sponsor.
Throughout, the tracker shows who contacted sponsor stands.

What NOT to Do
- ❌ Build a mass-mailer that sends the same
- ❌ Send emails automatically without human review and approval.
- ❌ Spam sponsors with unlimited follow-ups
- ❌ Use paid APIs or ignore free-tier rate limits.
- ❌ Fabricate company details, past sponsor
- ❌ Build a simple ChatGPT/Gemini wrapper that only writes email text while tracking stays manual.
- ❌ Lose track of who contacted which compathe same sponsor.
- ❌ Focus on UI polish while the actual outreach workflow stays manual.

Evaluation Focus

Solutions will be evaluated on:
- practical club utility
- quality and personalization of outreach
- accuracy of sponsor research and categorize
- reliability of tracking and follow-ups
- effective human-in-the-loop design
- smart use of free API limits
- technical implementation
- usability
The final solution should be something the club can realistically adopt for future sponsorship drives.